murmur: Spatio-Temporal Acoustic Monitoring for Predictive Maintenance

Matthew Park

August 2026

Abstract

Industrial acoustic monitoring promises to detect mechanical failure before it occurs, from sound alone. Murmur is a system built for that task: an array of microphones feeds a spatio-temporal graph neural network over an inverse-distance acoustic topology, whose per-timestep embeddings drive a continuous-time liquid network that forecasts time-to-failure, with a convolutional autoencoder scoring per-node anomalies against each microphone's own rolling baseline.

This paper reports an audit of that codebase and a substantially corrected system. The codebase is unusual in one respect that turns out to matter: it was built by three independent coding agents working concurrently in the same repository. That produced a specific and reproducible failure pattern. Two agents implemented acoustic localisation twice and an evaluation harness twice; one of each survived. Six commits of finished work — source localisation, public-dataset benchmarking, anomaly explainability, and a fault taxonomy — were stranded on an unmerged branch, and two further modules, webhook alerting and ONNX export, existed only as untracked files inside an agent worktree and would have been destroyed by a routine cleanup. An audit performed by one agent correctly identified eleven defects and, having been placed in a read-only role, fixed none of them; all eleven were still live.

The most consequential finding is not a crash. The system reported a time-to-failure F_1 of 1.000. That figure was an artifact of the data generator: fault severity was drawn from $U(0.5, 1.0)$ while healthy sequences sat below 0.1, leaving a 0.4-wide interval containing no examples, with the 0.5 decision threshold sitting inside it. The classes were separable by construction. The same draw confined the middle severity band to a sliver, starving the per-group conformal calibration that exists specifically to hold coverage in the high-risk strata — so the one guarantee aimed at the regime an operator acts in was the one silently disabled.

I close the gap, correct eleven defects with a regression test for each, salvage and integrate the stranded work, and re-measure. Under the corrected distribution, F_1 falls from 1.000 to 0.9672 and mean absolute error improves from 0.0608 to 0.0313 against a mean-predictor baseline of 0.2450. Conformal coverage lands at 0.89 against a nominal 0.90 with mean interval width falling from 0.190 to 0.119, and all three severity strata receive their own radius for the first time, ordered $0.048 < 0.074 < 0.105$ from normal to critical.

I then run the detector against recorded machine audio — the DCASE 2020 Task 2 pump set, four units, unsupervised, on the official split — and the synthetic result does not transfer. Mean AUC is 0.7064 against a published autoencoder baseline of 0.7289, and mean pAUC at p=0.1 is 0.4061 against 0.5999. The shortfall is concentrated in precisely the low-false-alarm regime a plant would operate in. Per-unit AUC ranges from 0.6404 to 0.8116, so a single-unit evaluation would have supported almost any conclusion. That is

the number I would quote, and it says a system scoring 0.9672 on its own simulator is at best comparable to a standard baseline on real audio.

Contents

1 Introduction

Rotating machinery announces its own failure. A bearing race that has begun to spall emits a high-frequency squeal with harmonics; a pump drawing below its required suction head produces broadband noise punctuated by the collapse of vapour bubbles; a rotor carrying lost balance weight modulates at shaft rate. These signatures precede functional failure by days or weeks, and a technician walking the floor with thirty years of experience can often hear them. The motivation for automating this is not that the task is impossible for humans but that it does not scale: a plant has thousands of machines and one walkthrough per shift.

Murmur addresses this with a fixed microphone array and three models in sequence. Audio is captured in half-second chunks, converted to log-mel spectrograms on the GPU, and buffered into temporal windows. A spatio-temporal graph neural network treats the array as a graph — microphones are nodes, inverse-distance acoustic coupling gives the edges — and produces a per-node embedding at every timestep. Those embeddings drive a closed-form continuous-time network (CfC) that integrates over real inter-frame intervals to forecast time-to-failure. In parallel, a convolutional autoencoder scores each frame against that microphone's own rolling baseline using a median/MAD robust z-score.

1.1 Motivation for this paper

This project has an unusual provenance. It was developed by three independent coding agents operating concurrently on the same repository, in separate git worktrees, without coordination between them. That arrangement produces a failure mode worth documenting, because it is not the one intuition predicts. The naive worry is merge conflicts. What actually happened was quieter:

- **Duplicated effort that resolved arbitrarily.** Two agents independently implemented time-difference-of-arrival source localisation, and two independently implemented an evaluation harness. In each case one landed on `main` and the other did not. The surviving localisation implementation was not the more sophisticated one; it was the one wired into the inference worker.
- **Work stranded rather than lost.** One agent's branch carried six commits — benchmarking against MIMII and DCASE, GCC-PHAT localisation, anomaly explainability, and a grounded fault taxonomy — that were never merged. Two modules, a webhook alerting router and an ONNX export path, were never committed at all. They existed only as untracked files inside a worktree directory that a routine `git worktree remove` would have deleted without warning.
- **An audit that fixed nothing.** A third agent audited the repository and correctly identified eleven defects. It had been assigned a read-only role to avoid editing files another session held open. Its report was accurate and its findings were never applied. All eleven were still live when I verified them against the tree.

None of these are merge conflicts. Version control was working exactly as designed throughout; the failure is that concurrent agents produce work whose *integration* nobody owns.

The second motivation is the more important one. The system reported an F_1 of 1.000 on time-to-failure classification. A perfect score on a hard problem should be read as evidence

about the measurement rather than the model, and it was: the label distribution had a hole in it, and the decision threshold sat in the hole.

1.2 Contributions

1. **A defect audit** of a concurrently-developed codebase, classifying eleven verified defects into those that stop the system emitting output, those that run silently while disabling a documented capability, and those that corrupt reported measurements (Section 3). Each is characterised by mechanism and closed with a regression test.

2. **Correction of the measurement protocol.** The synthetic label distribution contained a 0.4-wide interval with no examples, spanning the classification threshold. I identify the mechanism, close it, and show the effect on every reported figure (Sections 4 and 7).

3. **Repair of the uncertainty quantification.** Per-group conformal calibration was falling back to a global radius for two of three severity strata. I show this was a consequence of the same distributional defect compounded by an undersized calibration split, and restore group-conditional coverage (Sections 6 and 7).

4. **Recovery and integration of stranded work.** Roughly 2,000 lines across seven modules were salvaged from an unmerged branch and an uncommitted worktree, deduplicated against the implementations that had already landed, and — critically — wired into the runtime rather than left as dead code (Section 8).

5. **A verifiable artifact**: 452 automated tests, a Kafka integration test that genuinely executes rather than skipping, and infrastructure validated against the tooling that consumes it (Section 8).

1.3 What this paper does not claim

I do not claim that Murmur detects acoustic faults in industrial machinery. Every number reported here is measured on a synthetic generator, and a generator is a statement of the author's assumptions rather than evidence about the world. The corrected generator produces spatially-localised faults with inverse-square attenuation, per-fault spectral profiles, temporal onset ramps, and an ambient floor deliberately decoupled from the label — but a model that performs well against those assumptions has demonstrated only that it can recover the structure I put in.

I also do not claim the pre-correction figures were wrong in the sense of being miscomputed. F_1 was 1.000; the arithmetic was correct. The claim is that the quantity being computed did not measure what the surrounding prose said it measured, and that this is the more dangerous kind of error because nothing fails.

The corrected and uncorrected numbers are not comparable as model comparisons. They are measured on different distributions. Where both appear in this paper it is to quantify the effect of the correction, never to suggest one model is better than another.

Section 7.5 does evaluate on recorded machine audio, and that result carries its own limits. It covers one machine type (pump) from one corpus, and it exercises only the anomaly detector: DCASE is single-channel, so the graph network and the continuous-time forecaster — the two

components this architecture is actually distinguished by — contribute nothing to that number and remain unvalidated outside the simulator.

2 Related Work

2.1 Acoustic condition monitoring

Vibration analysis is the established technique for rotating machinery, with envelope analysis and cepstral methods used to isolate bearing defect frequencies. Airborne acoustic monitoring trades signal quality for installation cost: a microphone requires no contact with the machine and can cover several assets at once, at the price of reverberation, masking, and ambient noise. The DCASE challenge series established anomalous sound detection as a benchmark task, and the MIMII corpus provides recordings of pumps, fans, valves, and slide rails under normal and faulty operation at controlled signal-to-noise ratios.

2.2 Unsupervised anomaly detection

Fault data is scarce by construction — a well-maintained plant produces overwhelmingly normal recordings — so the dominant framing is unsupervised: model the normal manifold and score departures from it. Reconstruction-error autoencoders remain the standard baseline. Murmur follows this approach and adds a per-node robust baseline: each microphone is judged against its own recent history using a median and median absolute deviation rather than a mean and standard deviation, because a developing fault contaminates precisely the statistics used to detect it.

2.3 Graph neural networks for sensor arrays

Where sensors have spatial relationships, graph networks exploit them directly. Spatio-temporal GNNs interleave graph convolution with temporal modelling and are well established in traffic forecasting. The application here treats microphones as nodes and acoustic coupling as edges, so that a fault localised at one machine propagates through the graph with distance-dependent attenuation.

2.4 Continuous-time recurrent models

Liquid time-constant networks and their closed-form approximation (CfC) define hidden state through an ODE, which makes them natural for irregularly sampled data: the inter-arrival interval enters the dynamics rather than being assumed uniform. This matters for an edge deployment where network jitter makes the sampling interval genuinely variable, and it is the justification for the architecture choice — a justification that, as Section 3 records, the original tensor plumbing defeated.

2.5 Conformal prediction

Split conformal prediction provides finite-sample marginal coverage without assumptions on the model or the noise distribution. Mondrian (per-group) conformal extends this to conditional coverage within strata, which is the relevant variant here: coverage averaged over a population

dominated by healthy machines can look excellent while being poor exactly where decisions are made.

3 Defect Audit

This section documents defects verified against the tree rather than inferred from reports. Each was reproduced before being fixed and has a regression test afterwards; the existing 123-test suite passed throughout, which is the point of recording them.

3.1 Class 1: the system stops emitting

A single dead microphone silences the entire array. The window assembler required every node to have reported before releasing a graph snapshot, with no timeout, no eviction, and no degraded mode:

```
def is_complete(self) -> bool:
    if len(self._windows) < self.num_nodes:
        return False
    ...
    return (newest - oldest) <= self.staleness_tolerance
```

If one microphone drops out, the remaining three stream indefinitely and nothing is ever emitted. Worse, the staleness check cannot recover: a node that stops updating retains its last window forever, so the spread between newest and oldest never returns below tolerance and even the surviving microphones stop producing. No metric fired. The dashboard displayed "waiting for model predictions", which is visually identical to a healthy, quiet plant.

For a system whose entire premise is not missing events, going silent is the worst available failure mode. The assembler now evicts stale windows, releases a degraded snapshot once a quorum is present after a bounded wait, zero-fills absent nodes so the topology stays index-aligned, and omits them from telemetry so nothing is fabricated for a microphone that said nothing.

3.2 Class 2: silent defects

These executed without error while disabling a documented capability.

A facility-level forecast presented as four machine-level forecasts. One time-to-failure value and one embedding were computed from the pooled graph readout and copied into every node's payload. The dashboard's four per-node cards were therefore identical by construction — an operator reading four independent machine forecasts was reading one number four times. The ST-GNN now exposes per-node embeddings and each microphone receives a forecast derived from its own trajectory.

A write-only Kafka topic. Every spectrogram frame from every node was serialised, compressed, and shipped to a broker that no consumer subscribed to, with retention paid on all of it. The topic is now opt-in.

The rate limiter as a memory exhaustion vector. Buckets were keyed by API key or client address — both attacker-controlled on a public endpoint — trimmed within each key

but never across keys, and cleared only at shutdown. One request per spoofed source grew the map without bound. The control added to prevent denial of service enabled a different one.

A tensor cache keyed on memory addresses. The batched graph topology was cached under `edge_index.data_ptr()`, which is unique only while the tensor is alive. Once freed, the allocator may reissue that address to an unrelated tensor and the cache would return edges belonging to a graph that no longer exists. The probability is low and the symptom — a silently wrong topology — is severe. The cache now keys on shape and confirms tensor identity, holding a reference so the address cannot be recycled.

An unauthenticated metrics endpoint. `/metrics` published per-node anomaly counts, z-scores, and failure forecasts — a live map of which machines in the plant are failing — without authentication even when an API key was configured, on a LoadBalancer-exposed service.

3.3 Class 3: measurement defects

The label distribution had a hole in it. This is the finding that invalidates a reported number, and it is developed in full in Section 4.

At-least-once delivery was overstated. The module documented that "offsets advance only after downstream publish succeeds". The implementation enqueued messages, served already-completed delivery callbacks, and committed:

```
producer.poll(0) # serves callbacks that already finished
consumer.commit(...) # acknowledges data still in the client queue
```

Messages still in the librdkafka queue were unacknowledged when the offset advanced, so a crash in that window loses precisely the frames the comment promises to protect. Broker-side idempotence guards duplication, not this. The producer is now drained before offsets move, and a failed drain holds the batch for redelivery.

An integration test that could not fail. The Kafka fixture probed availability by constructing a producer and calling `flush()`. Neither connects — librdkafka dials lazily, and flushing an empty queue returns zero regardless — so the fixture always reported the broker as present and the test then failed on a missing message rather than skipping. In continuous integration the service was moreover started with no configuration at all, and readiness was probed by opening a TCP socket. An open port is not a broker serving metadata, so a half-started Kafka would have produced a green run that exercised nothing.

A degenerate z-score reaching the operator. When a rolling baseline has no spread, the detector multiplied the relative departure by 10^3, producing z-scores in the tens of thousands that were written to a Prometheus gauge and interpolated into a language model prompt verbatim. The magnitude is meaningless — the ratio is taken against an arbitrarily small median — so it now saturates at a documented constant, preserving sign, because a channel that went quiet is not a bearing failure.

3.4 Infrastructure

Three defects would each have prevented deployment and only one was visible. Bitnami withdrew its versioned tags from Docker Hub, so the Kafka image referenced by the development compose file, the continuous integration service, and the production Kubernetes StatefulSet no longer resolves. Only the CI reference failed loudly; the StatefulSet would have failed at

deploy time. All three now use the Apache project's image, which required translating every environment variable (the two images use different prefixes) and setting the log directory explicitly — the Apache image defaults it to `/tmp`, so the StatefulSet would otherwise have retained a healthy 20 GiB volume claim and still lost every unconsumed message on restart.

Separately, the test suite could not run in CI at all: a package was added without being registered for installation, and because CI invokes `pytest` directly rather than `python -m pytest` there is no working directory on `sys.path` to fall back on. Six modules failed at collection, which aborts the entire run — including the integration job, whose own test was deselected.

4 Data and Simulation

4.1 The generator

Each sequence is a window of 50 frames across 4 microphones with 64 mel bins. An ambient noise floor is drawn per sequence and *independently of the label*, so a loud healthy machine can out-energise a quiet degrading one; this removes total loudness as a shortcut. With probability p the sequence contains a fault, which is assigned a source node, one of three spectral profiles (bearing, cavitation, imbalance), and a severity. The fault contributes

$$c_{t,n,b} \;=\; s \cdot r_t \cdot m_t \cdot g_n \cdot \phi_b, \qquad g_n \;=\; \frac{1}{1 + d_{\mathrm{src},n}^2},$$

where s is severity, r_t a temporal onset ramp, m_t a slow modulation, g_n inverse-square attenuation from the source microphone, and ϕ_b a Gaussian envelope over mel bins. The spatial term is what gives the graph convolution something to learn: a fault at node 0 is loudest at node 0 and attenuates with distance.

4.2 The defect

Severity was drawn from $U(0.5, 1.0)$ for faulty sequences, while healthy sequences received a label from $U(0.0, 0.1)$. Measured over 2,000 sequences, the resulting distribution is:

Label interval	Count
$[0.0, 0.1)$	1718
$[0.1, 0.5)$	**0**
$[0.5, 0.6)$	64
$[0.6, 0.8)$	113
$[0.8, 1.0]$	105

The maximum healthy label is 0.0999 and the minimum faulty label is 0.5011: an empty interval 0.4012 wide. Classification metrics threshold at 0.5, which lies inside it. The two classes were linearly separable with a margin, and $\mathrm{F}_1 = 1.000$ followed for any model that could determine whether a fault signature was present at all — a far easier question than estimating how advanced it is.

This also distorted the severity strata used for conformal grouping. With bands at $[0, 0.33)$, $[0.33, 0.66)$, $[0.66, 1]$, the faulty draw could only reach *warning* through the sliver $[0.5, 0.66)$, giving 5.0% of sequences against 85.9% normal.

4.3 The correction

Severity is now drawn from $U(0.05, 1.0)$. This places examples on both sides of the decision threshold and overlaps the healthy band at the bottom, so a faintly degrading machine is genuinely difficult to distinguish from a healthy one — which is the regime predictive maintenance operates in. The fault rate rises to 0.40 and N to 4,000; both are properties of the simulator chosen so that per-stratum calibration is estimable, not claims about real fleets.

5 Method

The pipeline is four stages. Log-mel spectrograms are computed on GPU in micro-batches and buffered into per-node windows of 50 frames.

Spatio-temporal GNN. Per-microphone temporal self-attention with sinusoidal positional encoding runs first, so each microphone attends over its own history without leaking across the array. Attention is permutation-invariant without positional encoding, which for a model detecting temporal signatures is a correctness defect rather than a refinement. Graph convolution then runs at *every* timestep — $B \times S$ disjoint copies of the topology in one call — which is what makes the spatial convolution time-resolved, and is the dominant training cost. Readout produces both a pooled graph embedding and per-node embeddings (0.31 M parameters).

Liquid network. A CfC integrates the per-node embedding sequence over the measured inter-frame intervals (0.05 M parameters). The sequence form is essential and was the subject of an earlier defect: feeding a single pooled vector repeated across time gives a continuous-time model a constant signal and discards the entire justification for the architecture.

Anomaly scoring. A convolutional autoencoder reconstructs single frames; per-node reconstruction error is converted to a robust z-score against that node's rolling median and MAD. Group normalisation rather than batch normalisation, because the worker scores one frame at a time.

Grounded diagnosis. For flagged frames the per-bin reconstruction error is decomposed across frequency bands. Because the anomaly score *is* the mean squared reconstruction error, this decomposition is exact rather than an approximation of sensitivity: each band's share is literally its contribution. Bands are matched against a fault catalogue using a Bhattacharyya coefficient over normalised distributions, which prevents a broad signature (cavitation, 1–8 kHz) from winning every high-frequency diagnosis by covering more ground than a narrow one.

6 Uncertainty Quantification

The forecaster emits a sigmoid. Nothing in the training objective makes 0.7 mean "fails 70% of the time", and that is the number a maintenance planner would schedule an outage against. Split conformal prediction converts it to an interval with finite-sample marginal coverage without assumptions on the model or error distribution: hold out a calibration set, measure realised residuals, take a quantile.

Grouping matters more than the base method here. Marginal coverage averaged over a population that is 70% healthy can be excellent while coverage among critical forecasts is poor — and critical forecasts are the only ones anyone acts on. Mondrian conformal calibrates

within strata, requiring `min_group_size` samples per stratum before a group receives its own radius.

Two compounding defects prevented this from working. The label gap starved the middle band, and the calibration set — half of a 15% test split — was 75 samples. The *warning* stratum received 8. Both high-risk bands fell back to the global radius, so the guarantee held on the healthy majority and nowhere else. Enlarging the test split to 20% raises calibration to 400 samples; combined with the corrected label distribution, all three strata now qualify.

7 Experiments

7.1 Protocol

Identical architecture, optimiser, schedule, and seed (1337) throughout; 50 epochs with cosine annealing and early stopping on validation loss. The calibration set is half the test split, disjoint from both training and validation — the validation set is not exchangeable with unseen data because early stopping selected on it. Coverage is measured on the other half, which neither the model nor the calibrator has seen.

7.2 Effect of the correction

Run A is the original configuration. Run B closes the label gap and raises N. Run C additionally enlarges the calibration split.

	A (original)	**B** (gap closed)	**C** (final)
Sequences N	1000	4000	4000
Train/val/test	70/15/15	70/15/15	65/15/20
F_1	1.0000	0.9712	0.9672
Precision	1.0000	0.9593	0.9529
Recall	1.0000	0.9833	0.9818
MAE	0.0608	0.0375	0.0313
Baseline MAE	0.3255	0.2433	0.2450
Coverage (nominal 0.90)	0.933	0.890	0.890
Mean interval width	0.190	0.141	0.119
Calibration n	75	300	400
Strata with own radius	1 / 3	2 / 3	**3 / 3**

7.2.1 Interpretation

F_1 falling from 1.000 to 0.9672 is the headline result and it is an improvement in the measurement, not a regression in the model. Run A's perfect score reported that no example lay near the threshold. Run C's precision of 0.9529 and recall of 0.9818 describe a detector that now makes both false positives and misses on a task where the classes overlap — and that is a quantity with content.

MAE improves in the same direction the classification metric worsens ($0.0608 \rightarrow 0.0313$), which is initially counterintuitive and is explained by the baseline. The mean-predictor baseline

also falls ($0.3255 \rightarrow 0.2450$) because the label distribution is less extreme; the ratio of model to baseline improves from $5.4\times$ to $7.8\times$. The regression task became easier while the thresholded classification task became harder, because filling the gap moves mass toward the middle in both cases.

Coverage moves from 0.933 to 0.890 against a nominal 0.900. Run A was *over*-covering by 3.3 points, which is not a success: it means intervals were wider than the data required, and width is the cost an operator pays. Run C is within one point of nominal with intervals 37% narrower.

7.3 Group-conditional calibration

The fitted radii in Run C:

Stratum	Calibration n	Radius
normal	293	0.0478
warning	44	0.0741
critical	63	0.1054
global	400	0.0513

The ordering is the result worth noting. Uncertainty grows monotonically with severity: the model is precise about healthy machines and progressively less so as failure approaches. That is a physically sensible structure, and it is exactly what the grouping exists to discover — and could not previously express, because in Runs A and B the high-risk strata inherited a global radius of 0.051, roughly half the 0.105 that critical forecasts actually require. Under the original configuration, intervals on the most consequential predictions were approximately twice too narrow, and the marginal coverage figure of 0.933 concealed it.

7.4 An intermediate hypothesis that was wrong

Run B left the *warning* stratum at 26 samples, still below the threshold of 30. My initial explanation was that a sigmoid output trained under MSE compresses predictions toward the extremes, under-populating the middle band in prediction space — groups are formed on predictions, not labels, since labels are unavailable at inference. Measuring the bucket occupancy of the two distributions refuted this: labels gave 440/70/90 across normal, warning, and critical, and predictions gave 440/69/91. No compression exists. The shortfall was ordinary sampling variance in a 300-sample calibration half, and the correct remedy was a larger calibration set rather than a different model. I record this because the plausible mechanism and the actual one prescribed different fixes.

7.5 Recorded machine audio

Everything above measures Murmur against material Murmur generated. This subsection does not.

The DCASE 2020 Task 2 development set contains ten-second recordings of industrial machines under normal and anomalous operation, with a fixed official split: training data is normal-only by construction, since the task is unsupervised detection and a system that saw

labelled anomalies during training would not be reporting a comparable number. I evaluate on the pump subset — 3,349 normal training clips and 856 test clips (400 normal, 456 anomalous) across four physical units — training a fresh autoencoder per unit for 20 epochs at seed 1337.

Only the anomaly detector is under test here. DCASE audio is single-channel, so there is no microphone array, no graph, and no time-of-arrival structure: the ST-GNN and the liquid network are not exercised at all. What this measures is the autoencoder and the robust scorer, which is the component the DCASE baseline is also built from and therefore the only one that can be compared.

Unit	AUC	pAUC@10%	AP	Baseline AUC
pump/id_00	0.8116	0.6371	0.8996	0.7289
pump/id_02	0.6948	0.3811	0.7773	0.7289
pump/id_04	0.6787	0.2620	0.7153	0.7289
pump/id_06	0.6404	0.3441	0.7257	0.7289
Mean	**0.7064**	**0.4061**	0.7795	0.7289

Mean AUC of 0.7064 sits 0.0225 below the published DCASE 2020 autoencoder baseline of 0.7289. Mean pAUC at p=0.1 is 0.4061 against a baseline of 0.5999, a shortfall of 0.194 — and pAUC is the metric that matters operationally, because it integrates only over the low-false-positive region where a plant would actually set its threshold. A system that fires on ten percent of healthy pumps has already been muted by its operators.

Two things deserve emphasis. First, per-unit AUC spans 0.6404 to 0.8116; reporting id_00 alone would have suggested the system beats the baseline by eight points, and reporting id_06 alone that it loses by nine. Second, and more importantly, the same detector scores $F_1 = 0.9672$ on the corrected simulator. The simulator was made harder in Section 4 and remains far easier than a real pump. A generator is a statement of its author's assumptions, and this is the measurement of how much those assumptions were worth.

The comparison carries two caveats. The baseline figures are transcribed from the DCASE 2020 results and should be verified against the official page before being cited. More substantively, the harness compares each *unit* against the *machine-type* aggregate baseline, so the per-row deltas are not strictly like-for-like; only the mean row is a fair comparison, and only against the pump aggregate.

8 The Artifact

8.1 Recovered work

Roughly 2,000 lines were salvaged from an unmerged branch and an uncommitted worktree: a benchmark harness for MIMII, DCASE, and IMS; performance benchmarks; a scenario generator with lead-time events; spectral anomaly explainability; a fault taxonomy; a webhook alerting router with cooldown suppression; and ONNX export with INT8 quantisation.

Duplicates were resolved rather than merged. The stranded localisation module was discarded in favour of the implementation already wired into the worker; two independent ROC implementations were collapsed to one while preserving the benchmark variant's deliberate policy of reporting chance rather than raising on a degenerate split.

Salvage alone would have reproduced the defect the audit identified — modules that exist but nothing calls. The recovered components are integrated: the worker computes a spectral explanation for flagged frames, matches it against the taxonomy, and attaches both to telemetry, where the API states them in the generation prompt and in the templated fallback. This directly addresses a finding that the projector conditioning generated text on audio is a random projection until trained, so unconditioned prose was being presented as diagnosis. Alerting routes to Slack, PagerDuty, or a webhook, with all sinks empty by default: a monitoring system should not page anyone until asked.

One defect was found in the recovered code itself. The ONNX verification routine compared the exported graph against a *freshly initialised* model, so it could only pass when the tolerance was wide enough to be meaningless.

8.2 Verification

452 tests, executing in roughly 25 seconds. Every audit finding has a regression test, because all eleven were invisible to the suite that existed. Kafka integration is verified against a real broker rather than skipped: the readiness probe performs the same metadata round trip the test fixture does, so passing it means the test genuinely runs.

Infrastructure is validated against the tooling that consumes it rather than by inspection — manifests through `kubeconform` (14 resources, 0 invalid), the container image pulled and its user, healthcheck path, and default log directory confirmed directly, and the package layout checked by reproducing the Dockerfile's copy set in isolation.

9 Discussion

9.1 On perfect scores

The practical lesson is narrow and transferable: a perfect score is a measurement result, and the first hypothesis should concern the measurement. The defect here required no unusual insight to find — a histogram of the labels shows it immediately — but nothing prompted anyone to draw one, because the number was good and good numbers do not invite scrutiny. The system had a comprehensive test suite, and it passed. No test asserted anything about the distribution the tests were run against.

9.2 On concurrent agents

The failure pattern from three agents in one repository was not conflicting edits. It was unowned integration: duplicated work resolved by which branch happened to merge, finished work stranded on branches nobody merged, valuable work existing only as untracked files, and an accurate audit whose findings nobody applied. Each agent behaved reasonably in isolation. Every one of these is a coordination failure at the boundary rather than a defect inside any agent's work, and version control did not detect any of them because none is a conflict.

9.3 Limitations

Results are measured on a simulator and establish only that the pipeline recovers structure that was deliberately placed in it. The fault catalogue is small and its frequency bands are nominal

rather than derived from machine-specific kinematics. The array is four microphones in one room; the graph is small enough that the spatial convolution's contribution over a per-node MLP has not been isolated by ablation. The conformal guarantee assumes exchangeability between calibration and production, which acoustic drift breaks — a drift monitor exists to catch coverage collapse, but the guarantee is conditional on it. Alert escalation deliberately bypasses the suppression cooldown, so a signal oscillating between warning and critical will re-page on each upward transition; this is defensible but untested against real fault progressions.

10 Conclusion

I audited a spatio-temporal acoustic monitoring system built concurrently by three independent agents, verified eleven defects that a previous audit had correctly identified and left unfixed, and corrected them with regression tests. The most consequential finding was not a crash but a measurement: a reported F_1 of 1.000 that reflected a 0.4-wide hole in the label distribution spanning the decision threshold. Closing it moved F_1 to 0.9672 and, together with an enlarged calibration split, restored group-conditional conformal coverage across all three severity strata for the first time — revealing that intervals on critical forecasts had been approximately twice too narrow, concealed behind an apparently healthy marginal coverage figure.

On recorded pump audio the corrected detector reaches a mean AUC of 0.7064 against a published baseline of 0.7289, and a mean pAUC of 0.4061 against 0.5999. It does not beat a standard autoencoder baseline, and it is furthest behind in the low-false-alarm regime that determines whether an alerting system survives contact with the people it pages. I regard obtaining that number as the more valuable outcome than the number itself: the gap between 0.9672 on the simulator and 0.71 on real audio is the quantity the earlier reported $F_1 = 1.000$ was concealing, and it is now measured rather than assumed.

What this work provides is a pipeline whose claims can be checked: the measurement protocol is corrected, the uncertainty quantification does what it advertises in the regime that matters, and performance on real machine audio is established rather than asserted — including where it falls short.

The obvious next steps follow from the gap. The spatial and temporal components are untested on real data, because DCASE is single-channel and no public corpus provides a synchronised industrial microphone array; producing one, or evaluating on MIMII's multi-channel recordings, would exercise the parts of the architecture the pump result leaves entirely unmeasured. And the pAUC shortfall is the concrete target: a detector at 0.41 where the baseline reaches 0.60 has a specific, measurable deficiency in the only operating region that matters.